

Intermediate Code Generation

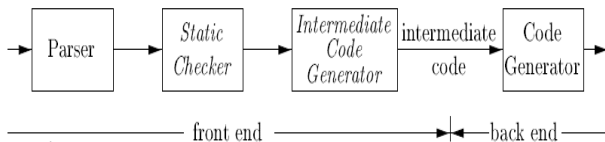
Compiler Design (CS 3007)

S. Pyne

Assistant Professor
Computer Science and Engineering Department
National Institute of Technology Rourkela
pynes@nitrkl.ac.in

April 14, 2026

Front and back ends



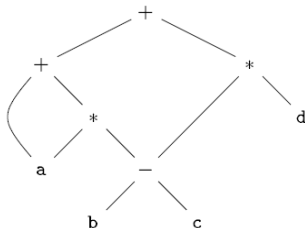
- Intermediate code
 - Machine independent representation of source code
 - Simplifies conversion from source code to target code
- Analysis synthesis - syntax and semantic analysis on source code
- Front end - intermediate code from source code
- Back end - target code from intermediate code
- Intermediate representation
 - Combines front end of i with back end of machine j
 - $m \times n$ compilers - by writing m front ends and n back ends

Implementation

- Traversal of syntax trees
- Three address code representation - $x = y \text{ op } z$
- Syntax tree - closer to high level language
- Three address code - range from high- to low-level
- Example - for loop statements
 - Syntax tree represents statement components
 - Three address code has labels and jump instruction
- Intermediate code representation
 - An actual programming language like C, or
 - Internal data structures shared by different phases
- Original C++ compiler
 - Front end generated intermediate code in C language
 - Back end compiled intermediate C code

Syntax trees

- Syntax nodes represent constructs in source program
- Node children represents components of a construct
- Directed Acyclic graph (*DAG*) for common sub-expression
- *DAG* for the expression $a + a * (b - c) + (b - c) * d$



- Sub-expression $b - c$ occur twice
- $b - c$ is represented by one node labeled $-$
- Node labeled $-$ has two parents for $a * (b - c)$ and $(b - c) * d$

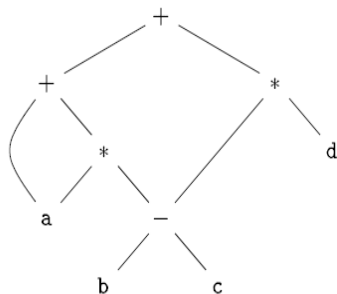
SDD for syntax tree and *DAG*

PRODUCTION	SEMANTIC RULES
1) $E \rightarrow E_1 + T$	$E.node = \text{new Node}('+', E_1.node, T.node)$
2) $E \rightarrow E_1 - T$	$E.node = \text{new Node}('-', E_1.node, T.node)$
3) $E \rightarrow T$	$E.node = T.node$
4) $T \rightarrow (E)$	$T.node = E.node$
5) $T \rightarrow \text{id}$	$T.node = \text{new Leaf}(\text{id}, \text{id.entry})$
6) $T \rightarrow \text{num}$	$T.node = \text{new Leaf}(\text{num}, \text{num.val})$

- Functions *Leaf* and *Node* create new nodes
- Creates *DAG* is if identical node already exists
- *Node*(*op*, *left*, *right*)
 - Checks a node labeled *op* with children *left* and *right*
 - If found *Node* returns existing node, otherwise creates a new node

DAG construction

- 1) $p_1 = \text{Leaf}(\text{id}, \text{entry-}a)$
- 2) $p_2 = \text{Leaf}(\text{id}, \text{entry-}a) = p_1$
- 3) $p_3 = \text{Leaf}(\text{id}, \text{entry-}b)$
- 4) $p_4 = \text{Leaf}(\text{id}, \text{entry-}c)$
- 5) $p_5 = \text{Node}('-', p_3, p_4)$
- 6) $p_6 = \text{Node}('*', p_1, p_5)$
- 7) $p_7 = \text{Node}('+', p_1, p_6)$
- 8) $p_8 = \text{Leaf}(\text{id}, \text{entry-}b) = p_3$
- 9) $p_9 = \text{Leaf}(\text{id}, \text{entry-}c) = p_4$
- 10) $p_{10} = \text{Node}('-', p_3, p_4) = p_5$
- 11) $p_{11} = \text{Leaf}(\text{id}, \text{entry-}d)$
- 12) $p_{12} = \text{Node}('*', p_5, p_{11})$
- 13) $p_{13} = \text{Node}('+', p_7, p_{12})$



$a + a * (b - c) + (b - c) * d$

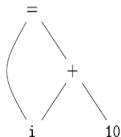
DAG construction - Value-Number Method

INPUT: Label op , node l , and node r .

OUTPUT: The value number of a node in the array with signature (op, l, r) .

METHOD: Search the array for a node M with label op , left child l , and right child r .

- ① If there is such a node, return the value number of M .
- ② If not, create in the array a new node N with label op , left child l , and right child r , and return its value number.



(a) DAG

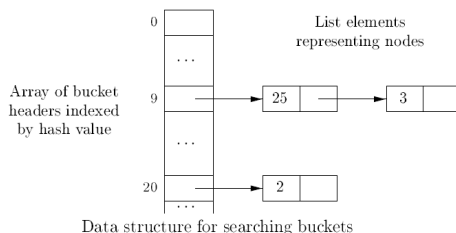
1	id			→ to entry for i
2	num		10	
3	+	1	2	
4	=	1	3	
5	...			

(b) Array

Nodes of a DAG for $i = i + 10$ allocated in an array

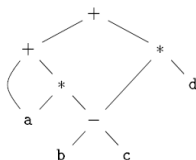
Data Structure - Value-Number Method

- Hash table for the nodes of a DAG
- Hash function $h(op, l, r)$ computes the index of the bucket for (op, l, r)
- $h(op, l, r)$ is computed deterministically from (op, l, r)



Three-address Code

- A linearized representation of a syntax tree or a DAG in which explicit names correspond to the interior nodes of the graph
- There is at most one operator on the right side of an instruction; that is, no built-up arithmetic expressions are permitted
- Three-address code for the expression: $a + a * (b - c) + (b - c) * d$



(a) DAG

```
t1 = b - c
t2 = a * t1
t3 = a + t2
t4 = t1 * d
t5 = t3 + t4
```

(b) Three-address code

A DAG and its corresponding three-address code

- $t_1, t_2, \dots, t_5$ are compiler-generated temporary names
- Target-code generated and code optimized from three-address code
- Intermediate value names allow rearrangement of three-address code

Three-address Code - Addresses

An address can be one of the following

- A name
 - source-program names to appear as addresses in three-address code
 - a source name is replaced by a pointer to its symbol-table entry, where all information about the name is kept
- A constant
- A compiler-generated temporary
 - useful in optimizing compilers
 - to create a distinct name each time a temporary is needed
 - temporaries can be combined when registers are allocated to variables

Three-address Code - Instructions

- ① Assignment instructions $x = y \text{ op } z$ for binary operations
- ② Assignments $x = \text{op } y$, for binary operations
- ③ Copy instructions $x = y$, x is assigned the value of y
- ④ An unconditional jump **goto** L
- ⑤ Conditional jumps **if** $x = \text{true}$ **goto** L and **if** $x = \text{false}$ **goto** L
- ⑥ Conditional jumps **if** $x \text{ relop } y$ **goto** L , $\text{relop} \in \{<, <=, ==, >=, >\}$
- ⑦ Procedure call and return for $p(x_1, x_2, \dots, x_n)$
 - Procedure call - **call** p, n
 - Function call - $y = \text{call } p, n$
 - Three address instructions
param x_1
param x_2
...
param x_n
call p, n

Three-address Code - Instructions

8 Indexed copy instructions

- $x = y[i]$ sets x to value in location i memory units beyond location y
- $x[i] = y$ sets contents of location i units beyond x to value of y

8 Address and pointer assignments

- $x = \&y$ sets the r-value of x to be the location (l-value) of y
- $x = *y$ - r-value of x is made equal to contents of location pointed to by y
- $*x = y$ sets the r-value of the object pointed to by x to the r-value of y
- l-value left side of assignments
- r-value right side of assignments
- l-value - location, r-value - content
- Variable x : l-value is memory location $\&x$, r-value is content at $\&x$
- Constant x : r-value is x , no l-value
- Pointer $*x$: l-value is $\&x$, r-value is location pointed to by x
- $x[i]$: l-value is $\&x[i]$ or $x + i$, r-value is $x[i]$ or $i[x]$ or $*(x + i)$
- Common “l-value required” errors: $5 = x$ and $x + y = z$

An Example

- Consider the statement
do $i = i+1$; while ($a[i] < v$);
- Two possible translations as follows

```
L:  t1 = i + 1  
    i = t1  
    t2 = i * 8  
    t3 = a [ t2 ]  
    if t3 < v goto L
```

(a) Symbolic labels.

```
100: t1 = i + 1  
101: i = t1  
102: t2 = i * 8  
103: t3 = a [ t2 ]  
104: if t3 < v goto 100
```

(b) Position numbers.

Two ways of assigning labels to three-address statements

Quadruples

- Three-address instructions do not specify representation of instruction in a data structure
- A quadruple (or quad) has four fields - *op*, *arg₁*, *arg₂*, and *result*
- *op* field contains an internal code for the operator
- Three-address instruction $x = y + z$ is represented by placing
 - $+$ in *op*,
 - y in *arg₁*, z in *arg₂*, and x in *result*
- Some exceptions
 - 1 Unary *op* ($x = \textit{minus } y$) or assignment *op* ($x = y$) do not use *arg₂*
 - 2 Operators like *param* use neither *arg₂* nor *result*
 - 3 Conditional and unconditional jumps put the target label in *result*

An Example

- Consider the assignment statement: $a = b * -c + b * -c;$
- Three-address code and its quadruple is

```
t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5
```

(a) Three-address code

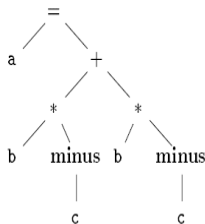
	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>	<i>result</i>
0	minus	c		t ₁
1	*	b	t ₁	t ₂
2	minus	c		t ₃
3	*	b	t ₃	t ₄
4	+	t ₂	t ₄	t ₅
5	=	t ₅		a
	...			

(b) Quadruples

- In general quadruple fields are pointers to their symbol-table entries
- Temporary names can be either
 - entered into symbol table like programmer-defined names, or
 - implemented as objects of a class *Temp* with its own methods

Triples

- A triple has three fields - *op*, *arg₁*, and *arg₂*
- In quadruples *result* field is primarily used as temporary names
- Triple refers to result of an operation $x \text{ op } y$ by its position
- t1 in following quadruple is referred as position (0) in triple



(a) Syntax tree

```

t1 = minus c
t2 = b * t1
t3 = minus c
t4 = b * t3
t5 = t2 + t4
a = t5
    
```

(b) Three-address code

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>	<i>result</i>
0	minus	c		t ₁
1	*	b	t ₁	t ₂
2	minus	c		t ₃
3	*	b	t ₃	t ₄
4	+	t ₂	t ₄	t ₅
5	=	t ₅		a
	...			

(c) Quadruples

	<i>op</i>	<i>arg₁</i>	<i>arg₂</i>
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
	...		

(d) Triples

Representations of $a = b * - c + b * - c$;

Indirect Triples

- In triples the result of an operation is referred to by its position
- Moving an instruction may change all references to that result
- Indirect triples solve this problem
- Indirect triples consist of a listing of pointers to triples
- Uses an array to list pointers to triples in the desired order

instruction

35	(0)
36	(1)
37	(2)
38	(3)
39	(4)
40	(5)
	...

op *arg₁* *arg₂*

0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)
		...	

Indirect triples representation of three-address code

- Indirect triples enable an optimizing compiler to move instructions by reordering the pointer list without affecting the triples
- Java implementation of indirect triples can be an array of references to objects of triple objects

Static Single-Assignment (SSA) Form

- SSA is an intermediate representation for certain code optimizations
- SSA differs from three-address code in two ways

- All assignments in SSA are to variables with distinct names - *static single-assignment*

```
p = a + b
q = p - c
p = q * d
p = e - p
q = p + q
```

```
p1 = a + b
q1 = p1 - c
p2 = q1 * d
p3 = e - p2
q2 = p3 + q1
```

(a) Three-address code. (b) Static single-assignment form.

- Subscripts distinguish each definition of variables p and q
- SSA uses ϕ -function to combine two definitions of a variable in two different control-flow paths in a source program

```
if ( flag ) x = -1; else x = 1;      if ( flag ) x1 = -1; else x2 = 1;
y = x * a;                          x3 =  $\phi(x_1, x_2)$ ;
```

(a) Two different flow paths in a program (b) ϕ -function to combine the two definitions of x

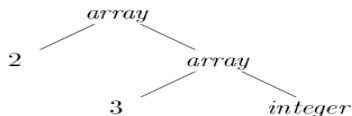
$$\phi(x_1, x_2) = \begin{cases} x_1, & \text{if flag is true} \\ x_2, & \text{if flag is false} \end{cases}$$

Types and Declarations

- Type checking
 - Uses rules to reason about the behavior of a program at run time
 - Ensures that types of operands match type expected by an operator
 - For example, the && operator in Java expects its two operands to be booleans; the result is also of type boolean
- Translation Applications
 - From type of a name, a compiler determines the storage needed by it at run time
 - Type information is needed to calculate the address denoted by an array reference
 - To insert explicit type conversions
 - To choose the right version of an arithmetic operator

Type Expressions

- A type expression is either a basic type or is formed by applying an operator called a type constructor to a type expression
- The sets of basic types and constructors depend on the language to be checked
- For example, the array type `int[2][3]`
 - can be read as “array of 2 arrays of 3 integers each”
 - written as a type expression `array(2, array(3, integer))`
 - This type is represented by the tree



Type expression for `int[2][3]`

- The operator *array* takes two parameters - a number and a type

Definition of type expressions

- A basic type is a type expression.
Basic types in a programming language - boolean, char, integer, float, and *void*
- A type name is a type expression.
- Type expression can be formed by applying array type constructor to a number and a type expression
- A type expression record is a data structure with named fields.
Formed by applying the record type constructor to the field names and their types.
- A type expression can be formed using type constructor $\rightarrow$ for function types. $s \rightarrow t$ denotes “function from type s to type t ”
- If s and t are type expressions, then their Cartesian product $s \times t$ is a type expression. Used to represent a list or tuple of types (e.g. for function parameters).
- Type expressions may contain variables whose values are type expressions.

Type Equivalence

- Two types are structurally equivalent iff one of the following is true
 - They are the same basic type
 - They are formed by applying the same constructor to structurally equivalent types
 - One is a type name that denotes the other
- For type names
first two conditions in above definition lead to *name equivalence* of type expressions

Declarations

- The following grammar deals with basic and array types

$$D \rightarrow T \text{ id}; D | \epsilon$$
$$T \rightarrow B \ C | \text{record } \{ 'D' \}$$
$$B \rightarrow \text{int} \mid \text{float}$$
$$C \rightarrow \epsilon \mid [\text{num}] \ C$$

- Nonterminal D generates a sequence of declarations
- Nonterminal T generates basic, array, or record types
- Nonterminal B generates one of basic types **int** and **float**
- Nonterminal C , for “component”, generates strings of zero or more integers, each integer surrounded by brackets
- An array type consists of a basic type specified by B , followed by array components specified by C
- A record type (the second production for T) is a sequence of declarations for the fields of the record, surrounded by curly braces

Storage Layout for Local Names

- Type of a name determines amount of storage it needs at run time
- At compile time, amount of storage assigns a relative address to each name. Type and relative address are saved in symbol-table entry for the name
- Data of varying length, such as strings or dynamic arrays whose size cannot be determined until run time are handled reserving fixed amount of storage for a pointer to the data
- Run-time storage management deals with data of varying length
- Consider storage in blocks of contiguous bytes where a byte is the smallest unit of addressable memory
- A byte is eight bits, and some number of bytes form a machine word
- Multibyte objects are stored in consecutive bytes and given the address of first byte
- Width of a type is number of storage units needed for objects of that type. A basic type, such as a **char**, **int**, or , **float** requires an integral number of bytes. For easy access, storage for aggregates such as arrays and classes is allocated in one contiguous block of bytes

Computing Storage for Local Names

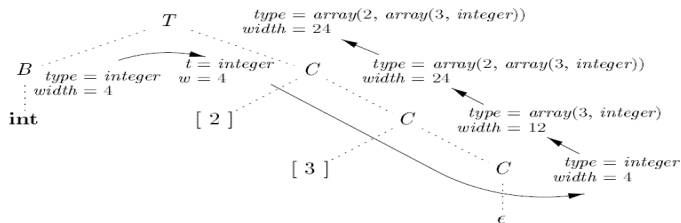
- SDT for computing type and width of basic and array types

$T \rightarrow B$	$\{ t = B.type; w = B.width; \}$
C	$\{ T.type = C.type; T.width = C.width; \}$
$B \rightarrow \text{int}$	$\{ B.type = \text{integer}; B.width = 4; \}$
$B \rightarrow \text{float}$	$\{ B.type = \text{float}; B.width = 8; \}$
$C \rightarrow \epsilon$	$\{ C.type = t; C.width = w; \}$
$C \rightarrow [\text{num}] C_1$	$\{ C.type = \text{array}(\text{num.value}, C_1.type);$ $C.width = \text{num.value} \times C_1.width; \}$

Computing types and their widths

- SDT uses synthesized attributes type and width for each nonterminal
- Variables t and w pass type and width information from a B node in a parse tree to the node for $C \rightarrow \epsilon$.
- Action between B and C sets t to $B.type$ and w to $B.width$
- If $B \rightarrow \text{int}$ then $B.type$ is set to integer and $B.width$ is set to 4
- If $B \rightarrow \text{float}$ then $B.type$ is set to float and $B.width$ is set to 8
- $C \rightarrow \epsilon$ then t becomes $C.type$ and w becomes $C.width$
- Otherwise, C specifies an array component

Syntax-directed translation of array types



- The parse tree for the type `int[2][3]` shown by dotted lines
- The solid lines show how the type and width are passed
- The variables t and w are assigned the values of $B.type$ and $B.width$
- Then the subtree with the C nodes is examined
- The values of t and w are used at the node for $C \rightarrow \epsilon$ to start evaluation of synthesized attributes up the chain of C nodes

Sequences of Declarations

- In C and Java declarations in a procedure is processed as a group
- Distributed declarations in Java procedure, processed during analysis
- Use a variable *offset* to keep track of next relative address
- Following SDT deals with a sequence of declarations of the form $T \text{ id}$

$$P \rightarrow D \quad \{ \text{offset} = 0; \}$$

$$D \rightarrow T \text{ id} ; \quad \{ \text{top.put}(\text{id.lexeme}, T.\text{type}, \text{offset}); \\ \text{offset} = \text{offset} + T.\text{width}; \}$$

$$D \rightarrow D_1 \epsilon$$

Computing the relative addresses of declared names

- Semantic action within $D \rightarrow T \text{ id} ; D_1$ creates a symbol-table entry
- *top* denotes current symbol table
- *top.put* creates a symbol-table entry for *id.lexeme* with type *T.type* and relative address *offset* in its data area

Fields in Records and Classes

- Field names within a record must be distinct
- Offset or relative address for a field name is relative to data area for that record

$$\begin{aligned} T \rightarrow \text{record } \{ & \{ \text{Env.push}(top); top = \text{new Env}(); \\ & \text{Stack.push}(offset); offset = 0; \} \\ D \} & \{ T.type = \text{record}(top); T.width = offset; \\ & top = \text{Env.pop}(); offset = \text{Stack.pop}(); \} \end{aligned}$$

- In the first action
 - The call *Env.push(top)* pushes the current symbol table denoted by *top* onto a stack
 - *top* is then set to a new symbol table
 - *offset* is pushed onto a stack called *Stack*
 - *offset* is then set to 0
- In the second action
 - *T.type* is set to *record(top)* and *T.width* is to *offset*
 - *top* and *offset* are then restored to their pushed values to complete the translation of this record type

Translation of Expressions

PRODUCTION	SEMANTIC RULES
$S \rightarrow \mathbf{id} = E ;$	$S.code = E.code \parallel$ $gen(top.get(\mathbf{id.lexeme}) '=' E.addr)$
$E \rightarrow E_1 + E_2$	$E.addr = \mathbf{new Temp}()$ $E.code = E_1.code \parallel E_2.code \parallel$ $gen(E.addr '=' E_1.addr '+' E_2.addr)$
$ - E_1$	$E.addr = \mathbf{new Temp}()$ $E.code = E_1.code \parallel$ $gen(E.addr '=' \mathbf{'minus'} E_1.addr)$
$ (E_1)$	$E.addr = E_1.addr$ $E.code = E_1.code$
$ \mathbf{id}$	$E.addr = top.get(\mathbf{id.lexeme})$ $E.code = ''$

- $S.code$ and $E.code$ denote three-address code for S and E
- $E.addr$ denotes the address that will hold the value of E

Incremental Translation

- A single sequence of instructions is created by successive calls to *gen*

$$\begin{aligned} S &\rightarrow \text{id} = E ; \quad \{ \text{gen}(\text{top.get}(\text{id.lexeme}) \text{'='} E.\text{addr}); \} \\ E &\rightarrow E_1 + E_2 \quad \{ E.\text{addr} = \text{new Temp}(); \\ &\quad \text{gen}(E.\text{addr} \text{'='} E_1.\text{addr} \text{'+'} E_2.\text{addr}); \} \\ E &\rightarrow E_1 * E_2 \quad \{ E.\text{addr} = \text{new Temp}(); \\ &\quad \text{gen}(E.\text{addr} \text{'='} E_1.\text{addr} \text{'*'} E_2.\text{addr}); \} \\ &| - E_1 \quad \{ E.\text{addr} = \text{new Temp}(); \\ &\quad \text{gen}(E.\text{addr} \text{'='} \text{'minus'} E_1.\text{addr}); \} \\ &| (E_1) \quad \{ E.\text{addr} = E_1.\text{addr}; \} \\ &| \text{id} \quad \{ E.\text{addr} = \text{top.get}(\text{id.lexeme}); \} \end{aligned}$$

- For $E \rightarrow E_1 + E_2$ calls *gen* to generate an add instruction
- Only after instructions computing E_1 and E_2 have already been generated

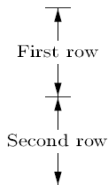
Example - Generating 3-address code using *gen*

	Input	Stack	<i>addr</i>	Generated Code
1.	A:=-B*(C+D)			
2.	:=B*(C+D)	id	A	
3.	:=B*(C+D)	id:=	A_	
4.	B*(C+D)	id:=	A__	
5.	*(C+D)	id:=id	A__B	
6.	*(C+D)	id:=E	A__B	T1:=-B
7.	*(C+D)	id:=E	A__T1	
8.	(C+D)	id:=E*	A__T1_	
9.	C+D)	id:=E*(	A__T1__	
10.	+D)	id:=E*(id	A__T1__C	
11.	+D)	id:=E*(E	A__T1__C	
12.	D)	id:=E*(E+	A__T1__C_	
13.	)	id:=E*(E+id	A__T1__C_D	
14.	)	id:=E*(E+E	A__T1__C_D	T2:=C+D
15.	)	id:=E*(E	A__T1__T2_	
16.	)	id:=E*(E)	A__T1__T2_	
17.	)	id:=E*E	A__T1__T2	T3:=T1*T2
18.	)	id:=E	A__T3	A:=T3
19.	)	A	A	

Addressing Array Elements

- Array elements in consecutive locations for quick access
- In C and Java, n array elements are indexed $0, 1, \dots, n - 1$
- Address of $A[i]$ is $base + i \times w$
 - $base$ is relative address of $A[0]$
 - w is width of each array element
- For 2 - d $A[n_1][n_2]$, address of $A[i_1][i_2]$ is $base + (i_1 \times n_2 + i_2) \times w$
- For $k - d$ $A[n_1][n_2] \dots [n_k]$, address of $A[i_1][i_2] \dots [i_k]$ is $base + ((\dots ((i_1 \times n_2 + i_2) \times n_3 + i_3) \dots) \times n_k + i_k) \times w$
- In general 1 - d array index numbers $low, low + 1, \dots, high$
 - Number of elements, $n = high - low + 1$
 - $base = addr(A[low])$
 - $addr(A[i]) = base + (i - low) \times w$
- Let $addr(A[i]) = i \times w + c$
- Compile time precalculation $c = base - low \times w$
- $c = base$ when $low = 0$
- c is saved in symbol table entry for A
- Hence using c from symbol table relative address of $A[i]$ is $i \times w + c$

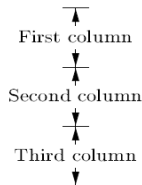
Layouts of 2-d Array



$A[1, 1]$
$A[1, 2]$
$A[1, 3]$
$A[2, 1]$
$A[2, 2]$
$A[2, 3]$

(a) Row Major

$A[1, 1]$
$A[2, 1]$
$A[1, 2]$
$A[2, 2]$
$A[1, 3]$
$A[2, 3]$



(b) Column Major

- Compile-time precalculation for addressing elements of multidimensional static arrays
- Compile-time precalculation not applicable to dynamic arrays
- c cannot be computed without knowing *low* and *high* in compile-time
- 2-d array stored in row-major (row-by-row) or column-major (column-by-column) form
- In general for multidimensional arrays
 - for row-major form rightmost subscript varies fastest while scanning a block of storage
 - for column-major form leftmost subscript varies fastest

Translation of Array References

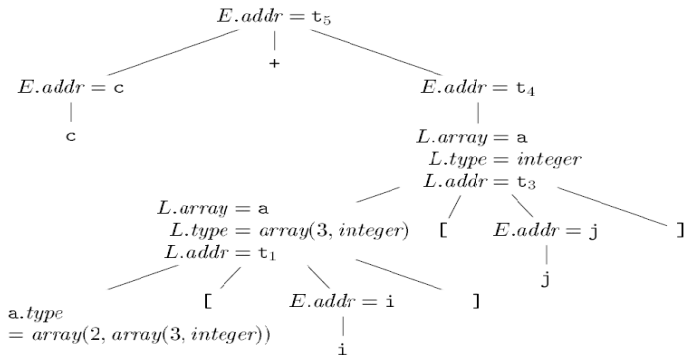
```

$$\begin{aligned} S &\rightarrow \mathbf{id} = E ; & \{ \text{gen}(\text{top.get}(\mathbf{id.lexeme}) \neq E.addr); \} \\ &| L = E ; & \{ \text{gen}(L.array.base '[ L.addr ]' \neq E.addr); \} \\ E &\rightarrow E_1 + E_2 & \{ E.addr = \mathbf{new Temp}(); \\ & & \text{gen}(E.addr \neq E_1.addr '+' E_2.addr); \} \\ &| \mathbf{id} & \{ E.addr = \text{top.get}(\mathbf{id.lexeme}); \} \\ &| L & \{ E.addr = \mathbf{new Temp}(); \\ & & \text{gen}(E.addr \neq L.array.base '[ L.addr ]'); \} \\ L &\rightarrow \mathbf{id} [ E ] & \{ L.array = \text{top.get}(\mathbf{id.lexeme}); \\ & & L.type = L.array.type.elem; \\ & & L.addr = \mathbf{new Temp}(); \\ & & \text{gen}(L.addr \neq E.addr '*' L.type.width); \} \\ &| L_1 [ E ] & \{ L.array = L_1.array; \\ & & L.type = L_1.type.elem; \\ & & t = \mathbf{new Temp}(); \\ & & L.addr = \mathbf{new Temp}(); \\ & & \text{gen}(t \neq E.addr '*' L.type.width); \\ & & \text{gen}(L.addr \neq L_1.addr '+' t); \} \end{aligned}$$

```

- $L.addr$ - offset for array reference
- $L.array$ - a pointer to the symbol-table entry for the array name
- $L.array.base$ - determines l-value of an array reference
- $L.type$ - type of the subarray generated by L
- $t.width$ - width of type t
- $t.elem$ - element type

Translation of Array References



Annotated parse tree for $c + a[i][j]$

```
t1 = i * 12
t2 = j * 4
t3 = t1 + t2
t4 = a [ t3 ]
t5 = c + t4
```

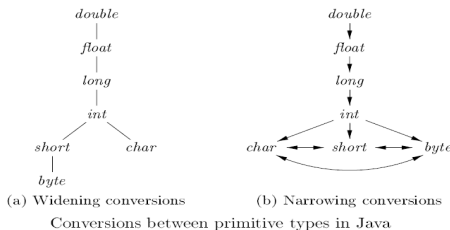
Three-address code for expression $c + a[i][j]$

Type Checking

- Type checking can take on two forms: synthesis and inference
- *Type synthesis* builds up type of an expression from types of its subexpressions
 - Type of $E_1 + E_2$ is defined in terms of types of E_1 and E_2
 - A typical rule for type synthesis has the form
if f has type $s \rightarrow t$ and x has type s ,
then expression $f(x)$ has type t
- *Type inference* determines type of a language construct from way it is used
 - $null(x)$ tells that x must be a list as empty list is null
 - A typical rule for type inference has the form
if $f(x)$ is an expression,
then for some α and β , f has type $\alpha \rightarrow \beta$ **and** x has type α
 - Type inference is needed for ML language, which check types, but do not declare names

Type Conversion

- How to solve expression $x + i$? - x is *float* and i is *int*
- Conversion from *int* to *float*: $t1 = (\text{float})\ 2$; $t2 = t1 * 3.14$;
- Conversion is *implicit* (*coercions*) if done by compiler
- Conversion is *explicit* (*casts*) if done by programmer
- In Java, *widening* conversions preserve information and *narrowing* conversions can lose information



- The semantic action for checking $E \rightarrow E_1 + E_2$ uses two functions
 - 1 $\max(t_1, t_2)$ returns maximum of types t_1 and t_2 in widening hierarchy
 - 2 $\text{widen}(a, t, w)$ returns address a if types t and w are same, otherwise it generates an instruction to convert

widen for *int* and *float*

- Generates an instruction to do the conversion
- Places the result in a temporary

```
Addr widen(Addr a, Type t, Type w)  
    if ( t = w ) return a;  
    else if ( t = integer and w = float ) {  
        temp = new Temp();  
        gen(temp '=' '(float)' a);  
        return temp;  
    }  
    else error;  
}
```

Pseudocode for function *widen*

Type Conversions into Expression Evaluation

$$E \rightarrow E_1 + E_2 \quad \left\{ \begin{array}{l} E.type = \max(E_1.type, E_2.type); \\ a_1 = \text{widen}(E_1.addr, E_1.type, E.type); \\ a_2 = \text{widen}(E_2.addr, E_2.type, E.type); \\ E.addr = \text{new Temp}(); \\ \text{gen}(E.addr \text{'=' } a_1 \text{'+' } a_2); \end{array} \right\}$$

- Temporary variable a_1 is either
 - $E_1.addr$, if the type of E_1 is not converted to type of E , or
 - a new temporary variable returned by *widen*, if converted
- Similarly, a_2 is $E_2.addr$ if not converted, otherwise new temp. var.
- In general, to add two different types convert both to a third type

Overloading of Functions and Operators

- An overloaded symbol has different meanings depending on its context
- Overloading is resolved when a unique meaning is determined for each occurrence of a name
- The $+$ operator in Java denotes either string concatenation or addition, depending on the types of its operands
- User-defined functions can be overloaded, as in

```
void err() { ... }  
void err(String s) { ... }
```
- Type-synthesis rule for overloaded functions
if f can have type $s_i \rightarrow t_i$, for $1 \leq i \leq n$, where $s_i \neq s_j$ for $i \neq j$
and x has type s_k , for some $1 \leq k \leq n$
then expression $f(x)$ has type t_k

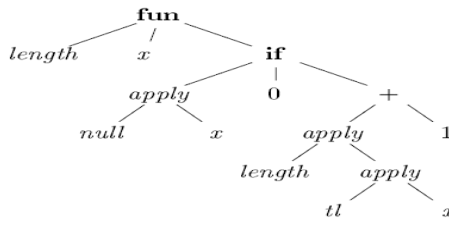
Type Inference and Polymorphic Functions

- Polymorphic - any code fragment executed with arguments of different types
- Parametric polymorphism by parameters or type variables
- Example - ML program for finding length of a list

fun *length*(*x*) =

if *null*(*x*) **then** 0 **else** *length*(*tl*(*x*)) + 1;

- *length*(["sun", "mon", "tue"]) + *length*([10, 9, 8, 7])
evaluates to $3 + 4 = 7$
- Abstract syntax tree for the function definition



Algorithm - Type Inference for Polymorphic Functions

- INPUT
- A program with sequence of function definitions
 - Followed by an expression to be evaluated
 - An expression has function applications and names
 - Names can have predefined polymorphic types

OUTPUT Inferred types for the names in the program

METHOD For simplicity, considering unary functions only

- Type of function $f(x_1, x_2)$ represented by $s_1 \times s_2 \rightarrow t$
- s_1 and s_2 are the types of x_1 and x_2 , respectively
- t is the type of the result $f(x_1, x_2)$
- An expression $f(a, b)$ is checked by matching type of a with s_1 and type of b with s_2

Algorithm - Type Inference for Polymorphic Functions

Check the function definitions and the expression in the input sequence. Use the inferred type of a function if it is subsequently used in an expression.

- For a function definition **fun** $\mathbf{id}_1(\mathbf{id}_2) = E$, create fresh type variables α and β . Associate the type $\alpha \rightarrow \beta$ with the function $\mathbf{id}_1$, and the type α with the parameter $\mathbf{id}_2$. Then, infer a type for expression E . Suppose α denotes type s and β denotes type t after type inference for E . The inferred type of function $\mathbf{id}_1$ is $s \rightarrow t$. Bind any type variables that remain unconstrained in $s \rightarrow t$ by $\forall$ quantifiers.
- For a function application $E_1(E_2)$, infer types for E_1 and E_2 . Since E_1 is used as a function, its type must have the form $s \rightarrow s'$. (Technically, the type of E_1 must unify with $\beta \rightarrow \gamma$, where β and γ are new type variables). Let t be the inferred type of E_2 . Unify s and t . If unification fails, the expression has a type error. Otherwise, the inferred type of $E_1(E_2)$ is s' .
- For each occurrence of a polymorphic function, replace the bound variables in its type by distinct fresh variables and remove the $\forall$ quantifiers. The resulting type expression is the inferred type of this occurrence.
- For a name that is encountered for the first time, introduce a fresh variable for its type.

Inferring a type for function *length*

LINE	EXPRESSION : TYPE	UNIFY
1)	$length : \beta \rightarrow \gamma$	
2)	$x : \beta$	
3)	$if : boolean \times \alpha_i \times \alpha_i \rightarrow \alpha_i$	
4)	$null : list(\alpha_n) \rightarrow boolean$	
5)	$null(x) : boolean$	$list(\alpha_n) = \beta$
6)	$0 : integer$	$\alpha_i = integer$
7)	$+ : integer \times integer \rightarrow integer$	
8)	$tl : list(\alpha_t) \rightarrow list(\alpha_t)$	
9)	$tl(x) : list(\alpha_t)$	$list(\alpha_t) = list(\alpha_n)$
10)	$length(tl(x)) : \gamma$	$\gamma = integer$
11)	$1 : integer$	
12)	$length(tl(x)) + 1 : integer$	
13)	$if(\dots) : integer$	

An Algorithm for Unification

$$((\alpha_1 \rightarrow \alpha_2) \times \text{list}(\alpha_3)) \rightarrow \text{list}(\alpha_2)$$

$$((\alpha_3 \rightarrow \alpha_4) \times \text{list}(\alpha_3)) \rightarrow \alpha_5$$

(a) Two type expressions

x	$S(x)$
-----	--------

α_1	α_1
------------	------------

α_2	α_2
------------	------------

α_3	α_1
------------	------------

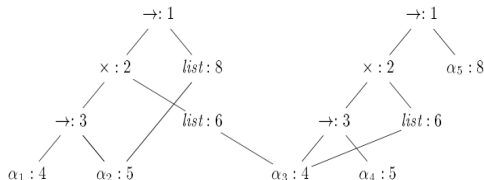
α_4	α_2
------------	------------

α_5	$\text{list}(\alpha_2)$
------------	-------------------------

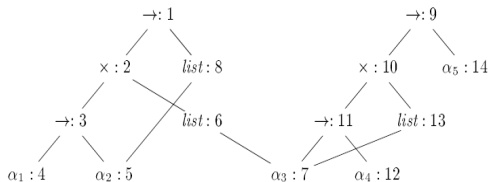
(b) Substitution S

$$((\alpha_1 \rightarrow \alpha_2) \times \text{list}(\alpha_1)) \rightarrow \text{list}(\alpha_2)$$

(c) Expression formed after substitution



(d) Equivalence classes after unification



(e) Initial graph with each node in its own equivalence class

An Algorithm for Unification

```
boolean unify(Node m, Node n) {  
    s = find(m); t = find(n);  
    if ( s = t ) return true;  
    else if ( nodes s and t represent the same basic type ) return true;  
    else if ( s is an op-node with children s1 and s2 and  
              t is an op-node with children t1 and t2 ) {  
        union(s, t);  
        return unify(s1, t1) and unify(s2, t2);  
    }  
    else if ( s or t represents a variable ) {  
        union(s, t);  
        return true;  
    }  
    else return false;  
}
```

Control Flow

- Control flow statements - if-else-statements and while-statements
- Translation of control flow statements is tied to translation of boolean expressions
- In programming languages, boolean expressions are used to
 - Alter the flow of control
 - Compute logical values

Boolean Expressions

- Grammar rules - $B \rightarrow B || B | B \& \& B | ! B | (B) | E \text{ rel } E | \text{true} | \text{false}$
- $rel.op \in \{<, <=, =, !=, >, >=\}$
- Precedence - ! highest, || lowest
- Associativity - ! is right, others are left

Short-Circuit Code

```
if ( x < 100 || x > 200 && x != y ) x = 0;
```

(a) Statement

```
if x < 100 goto L2
```

```
ifFalse x > 200 goto L1
```

```
ifFalse x != y goto L1
```

```
L2: x = 0
```

```
L1:
```

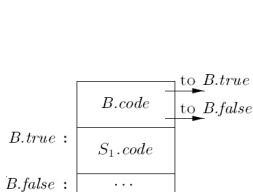
(b) Jumping code

- For logical OR - evaluation of expressions until an expression is true
- For logical AND - evaluation of expressions takes until an expression is false

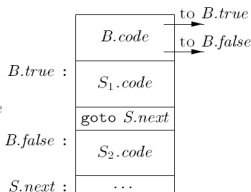
Flow-of-Control Statements

$$\begin{aligned} S &\rightarrow \text{if } (B) S_1 \\ S &\rightarrow \text{if } (B) S_1 \text{ else } S_2 \\ S &\rightarrow \text{while } (B) S_1 \end{aligned}$$

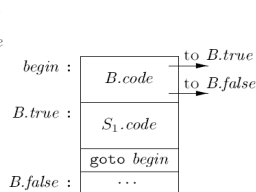
(a) Grammar for if-, if-else- and while-statements



(b) Code for if-statement



(c) Code for if-else-statement



(d) Code for while-statement

- *B* represents a boolean expression
- *S* represents a statement
- Translation of **if** (*B*)
 - consists of *B.code* followed by *S₁.code*
 - in *B.code* jumps are based on value of *B*
 - if *B* is true, control flows to first instruction of *S₁.code*
 - if *B* is false, control flows to instruction immediately following *S₁.code*

SDD for Flow-of-Control Statements

PRODUCTION	SEMANTIC RULES
$P \rightarrow S$	$S.next = newlabel()$ $P.code = S.code \parallel label(S.next)$
$S \rightarrow \text{assign}$	$S.code = \text{assign.code}$
$S \rightarrow \text{if} (B) S_1$	$B.true = newlabel()$ $B.false = S_1.next = S.next$ $S.code = B.code \parallel label(B.true) \parallel S_1.code$
$S \rightarrow \text{if} (B) S_1 \text{ else } S_2$	$B.true = newlabel()$ $B.false = newlabel()$ $S_1.next = S_2.next = S.next$ $S.code = B.code$ $\quad \parallel label(B.true) \parallel S_1.code$ $\quad \parallel gen('goto' S.next)$ $\quad \parallel label(B.false) \parallel S_2.code$
$S \rightarrow \text{while} (B) S_1$	$begin = newlabel()$ $B.true = newlabel()$ $B.false = S.next$ $S_1.next = begin$ $S.code = label(begin) \parallel B.code$ $\quad \parallel label(B.true) \parallel S_1.code$ $\quad \parallel gen('goto' begin)$
$S \rightarrow S_1 S_2$	$S_1.next = newlabel()$ $S_2.next = S.next$ $S.code = S_1.code \parallel label(S_1.next) \parallel S_2.code$

- $newlabel()$ creates a new label each time it is called
- $label(L)$ attaches label L to the next three-address instruction

Control-Flow Translation of Boolean Expressions

PRODUCTION	SEMANTIC RULES
$B \rightarrow B_1 \ \ B_2$	$B_1.true = B.true$ $B_1.false = newlabel()$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \ \ label(B_1.false) \ \ B_2.code$
$B \rightarrow B_1 \ \&\& \ B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \ \ label(B_1.true) \ \ B_2.code$
$B \rightarrow ! B_1$	$B_1.true = B.false$ $B_1.false = B.true$ $B.code = B_1.code$
$B \rightarrow E_1 \ \mathbf{rel} \ E_2$	$B.code = E_1.code \ \ E_2.code$ $\quad \ \ gen('if' \ E_1.addr \ \mathbf{rel.op} \ E_2.addr \ 'goto' \ B.true)$ $\quad \ \ gen('goto' \ B.false)$
$B \rightarrow \mathbf{true}$	$B.code = gen('goto' \ B.true)$
$B \rightarrow \mathbf{false}$	$B.code = gen('goto' \ B.false)$

Generation three-address code for Boolean Expressions

```
if( x < 100 || x > 200 && x != y ) x = 0;
```

(a) A Boolean Expression

```
      if x < 100 goto L2
      goto L3
L3:   if x > 200 goto L4
      goto L1
L4:   if x != y goto L2
      goto L1
L2:   x = 0
L1:
```

(b) Three-address code

Avoiding Redundant Gotos

```
if( x < 100 || x > 200 && x != y ) x = 0;
```

(a) A Boolean expression

```
    if x < 100 goto L2
    goto L3
L3:  if x > 200 goto L4
      goto L1
L4:  if x != y goto L2
      goto L1
L2:  x = 0
L1:
```

(b) Three-address code

```
    if x > 200 goto L4
    goto L1
```

L₄: ...

(c) Instruction for x > 200
has redundant gotos

```
    ifFalse x > 200 goto L1
```

L₄: ...

(d) Instruction for x > 200
without redundant gotos

- ifFalse instruction takes advantage of the natural flow from one instruction to the next in sequence
- Control simply “falls through” to label L₄ if x > 200, thereby avoiding a jump

Boolean Values and Jumping Code

- A boolean expression may also be evaluated for its value, as in assignment statements such as $x = \text{true}$; or $x = a < b$;
- Two translation approaches
 - 1 *Use two passes*
Construct a complete syntax tree for the input, and then walk the tree in depth-first order, computing the translations specified by the semantic rules.
 - 2 *Use one pass for statements, but two passes for expressions*
Translate E in **while** (E) S_1 before S_1 is examined. The translation of E is done by building its syntax tree and then walking the tree.

Backpatching

- A key problem when generating code for boolean expressions and flow-of-control statements is that of matching a jump instruction with the target of the jump
- For example, the translation of the boolean expression B in **if** (B) S contains a jump, for when B is false, to the instruction following the code for S .
- In a one-pass translation, B must be translated before S is examined
- What then is the target of the **goto** that jumps over the code for S ?
- This problem is addressed by passing labels as inherited attributes to where the relevant jump instructions were generated
- A separate pass is then needed to bind labels to addresses
- Backpatching - One pass alternative
 - Lists of jumps are passed as synthesized attributes
 - For jump where target of jump is temporarily left unspecified
 - Each such jump is put on a list of jumps whose labels are to be filled in when the proper label can be determined
 - All of the jumps on a list have the same target label

Backpatching for Boolean Expressions

- Maintain lists of jumps that are filled in when their targets are found
- B have two lists of jumps - $B.truelist$ and $B.falselist$
- Unfilled jumps attributed by $nextlist$
- $nextlist$ is completed by backpatching
- Jump list manipulation
 - $makelist(i)$ creates a new list containing only i , an index into the array of instructions and returns a pointer to the newly created list
 - $merge(p_1, p_2)$ concatenates lists pointed to by p_1 and p_2 , and returns a pointer to the concatenated list.
 - $backpatch(p_i)$ inserts i as the target label for each of the instructions on the list pointed to by p

Backpatching for Boolean Expressions

$$B \rightarrow B_1 \mid\mid M\ B_2 \mid B_1 \ \&\&\ M\ B_2 \mid !\ B_1 \mid (B_1) \mid E_1 \ \mathbf{rel}\ E_2 \mid \mathbf{true} \mid \mathbf{false}$$
$$M \rightarrow \epsilon$$

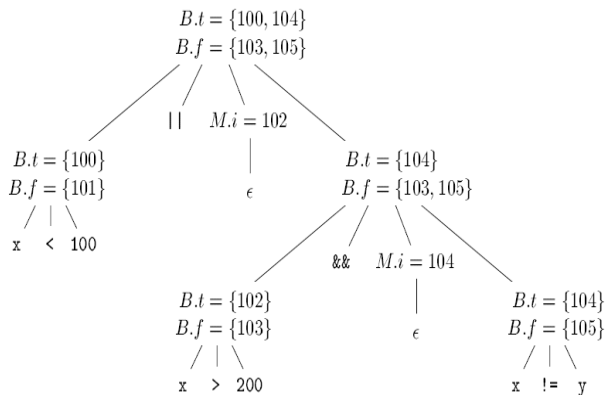
(a) Grammar rules

- 1) $B \rightarrow B_1 \mid\mid M\ B_2$ { *backpatch*(*B*₁.*false*list, *M.instr*);
 *B.true*list = *merge*(*B*₁.*true*list, *B*₂.*true*list);
 *B.false*list = *B*₂.*false*list; }
- 2) $B \rightarrow B_1 \ \&\&\ M\ B_2$ { *backpatch*(*B*₁.*true*list, *M.instr*);
 *B.true*list = *B*₂.*true*list;
 *B.false*list = *merge*(*B*₁.*false*list, *B*₂.*false*list); }
- 3) $B \rightarrow !\ B_1$ { *B.true*list = *B*₁.*false*list;
 *B.false*list = *B*₁.*true*list; }
- 4) $B \rightarrow (B_1)$ { *B.true*list = *B*₁.*true*list;
 *B.false*list = *B*₁.*false*list; }
- 5) $B \rightarrow E_1 \ \mathbf{rel}\ E_2$ { *B.true*list = *makelist*(*nextinstr*);
 *B.false*list = *makelist*(*nextinstr* + 1);
 gen('if' *E*₁.*addr rel.op E*₂.*addr goto _*');
 gen('goto _'); }
- 6) $B \rightarrow \mathbf{true}$ { *B.true*list = *makelist*(*nextinstr*);
 gen('goto _'); }
- 7) $B \rightarrow \mathbf{false}$ { *B.false*list = *makelist*(*nextinstr*);
 gen('goto _'); }
- 8) $M \rightarrow \epsilon$ { *M.instr* = *nextinstr*; }

(b) Translation scheme

• Backpatching to translate Boolean expressions in one pass

Annotated parse tree of a Boolean Expression



Annotated parse tree for $x < 100 \parallel x > 200 \&\& x \neq y$

Flow-of-control Statements

$$S \rightarrow \text{if}(B) S \mid \text{if}(B) S \text{ else } S \mid \text{while}(B) S \mid \{ L \} \mid A ;$$
$$L \rightarrow L S \mid S$$

(a) Grammar for flow-of-control statements

```
100:  if x < 100 goto _
101:  goto _
102:  if x > 200 goto 104
103:  goto _
104:  if x != y goto _
105:  goto _
```

```
100:  if x < 100 goto _
101:  goto 102
102:  if x > 200 goto 104
103:  goto _
104:  if x != y goto _
105:  goto _
```

(a) After backpatching 104 into instruction 102. (b) After backpatching 102 into instruction 101.

- Backpatching to translate flow-of-control statements in one pass

Backpatching of flow-of-control statements

- | | |
|--|---|
| 1) $S \rightarrow \text{if}(B) M S_1 \{$
$\text{backpatch}(B.\text{truelist}, M.\text{instr});$
$S.\text{nextlist} = \text{merge}(B.\text{falselist}, S_1.\text{nextlist}); \}$ | 4) $S \rightarrow \{ L \}$ $\{ S.\text{nextlist} = L.\text{nextlist}; \}$ |
| 2) $S \rightarrow \text{if}(B) M_1 S_1 N \text{ else } M_2 S_2$
$\{ \text{backpatch}(B.\text{truelist}, M_1.\text{instr});$
$\text{backpatch}(B.\text{falselist}, M_2.\text{instr});$
$\text{temp} = \text{merge}(S_1.\text{nextlist}, N.\text{nextlist});$
$S.\text{nextlist} = \text{merge}(\text{temp}, S_2.\text{nextlist}); \}$ | 5) $S \rightarrow A;$ $\{ S.\text{nextlist} = \text{null}; \}$ |
| 3) $S \rightarrow \text{while } M_1 (B) M_2 S_1$
$\{ \text{backpatch}(S_1.\text{nextlist}, M_1.\text{instr});$
$\text{backpatch}(B.\text{truelist}, M_2.\text{instr});$
$S.\text{nextlist} = B.\text{falselist};$
$\text{gen}('goto' M_1.\text{instr}); \}$ | 6) $M \rightarrow \epsilon$ $\{ M.\text{instr} = \text{nextinstr}; \}$ |
| | 7) $N \rightarrow \epsilon$ $\{ N.\text{nextlist} = \text{makelist}(\text{nextinstr});$
$\text{gen}('goto -'); \}$ |
| | 8) $L \rightarrow L_1 M S$ $\{ \text{backpatch}(L_1.\text{nextlist}, M.\text{instr});$
$L.\text{nextlist} = S.\text{nextlist}; \}$ |
| | 9) $L \rightarrow S$ $\{ L.\text{nextlist} = S.\text{nextlist}; \}$ |

- Maintain lists of jumps that are filled in when their targets are found
- B have two lists of jumps - $B.\text{truelist}$ and $B.\text{falselist}$
- Statements generated by S and L have a list of unfilled jumps
- Unfilled jumps attributed by nextlist
- nextlist is completed by backpatching
- $S.\text{nextlist}$ is a list of jumps follow code of S in execution order
- $L.\text{nextlist}$ is a list of jumps follow code of L in execution order

Break-, Continue- and Goto- Statements

```
1) for ( ; ; readch() ) {  
2)     if( peek == ' ' || peek == '\t' ) continue;  
3)     else if( peek == '\n' ) line = line + 1;  
4)     else break;  
5) }
```

- Generate code for the break by
 - ① keeping track of the enclosing loop statement *S*
 - ② generating an unfilled jump for the break-statement
 - ③ putting this unfilled jump on *S.nextlist*
- Continue-statements can be handled in a manner analogous to the break-statement
- Main difference between the two is that the target of the generated jump is different

Translation of Switch-Statements

<pre> switch (E) { case V_1: S_1 case V_2: S_2 ... case V_{n-1}: S_{n-1} default: S_n } </pre>	<pre> code to evaluate E into t goto test code for S_1 goto next code for S_2 goto next ... code for S_{n-1} goto next </pre>	<pre> code to evaluate E into t if $t \neq V_1$ goto L_1 code for S_1 goto next if $t \neq V_2$ goto L_2 code for S_2 goto next ... if $t \neq V_{n-1}$ goto L_{n-1} code for S_{n-1} goto next code for S_n next: </pre>
(a) Switch-statement syntax	<pre> L_{n-1}: code for S_{n-1} goto next L_n: code for S_n goto next test: if $t = V_1$ goto L_1 if $t = V_2$ goto L_2 ... if $t = V_{n-1}$ goto L_{n-1} goto L_n next: </pre>	<pre> L_1: code for S_1 goto next L_2: ... if $t \neq V_{n-1}$ goto L_{n-1} code for S_{n-1} goto next L_{n-1}: code for S_n next: </pre>
(b) Case three-address-code	(c) Translation of a switch-statement	(d) Another translation

- For each case new label L_i is created
- Each value-label pair V_i, L_i is stored in a queue
- Value-label pair acts as pointer to the symbol-table entry for L_i
- case $t \ V_i \ L_i$ instruction is a synonym for if $t=V_i$ goto L_i

Intermediate Code for Procedures

$D \rightarrow \text{define } T \text{ id } (F) \{ S \}$

$F \rightarrow \epsilon \mid T \text{ id } , F$

$S \rightarrow \text{return } E ;$

$E \rightarrow \text{id } (A)$

$A \rightarrow \epsilon \mid E , A$

(a) Rules for procedure

$n = f(a[i]);$

(b) A function call

1) $t_1 = i * 4$

2) $t_2 = a [t_1]$

3) **param** t_2

4) $t_3 = \text{call } f, 1$

5) $n = t_3$

(c) Three-address code

- Function types - return type and types of formal parameters
- Symbol tables - save status of current symbol table and create a new symbol table for function
- Type checking - checking parameters of formal and actual parameters
- Function calls - three-address instructions for a function call
id($E, E, \dots, E$) generate three-address instructions for evaluation and reduction of parameters E to addresses, followed by a **param** instruction for each parameter

Thank you